

Document Loaders & Text Splitting

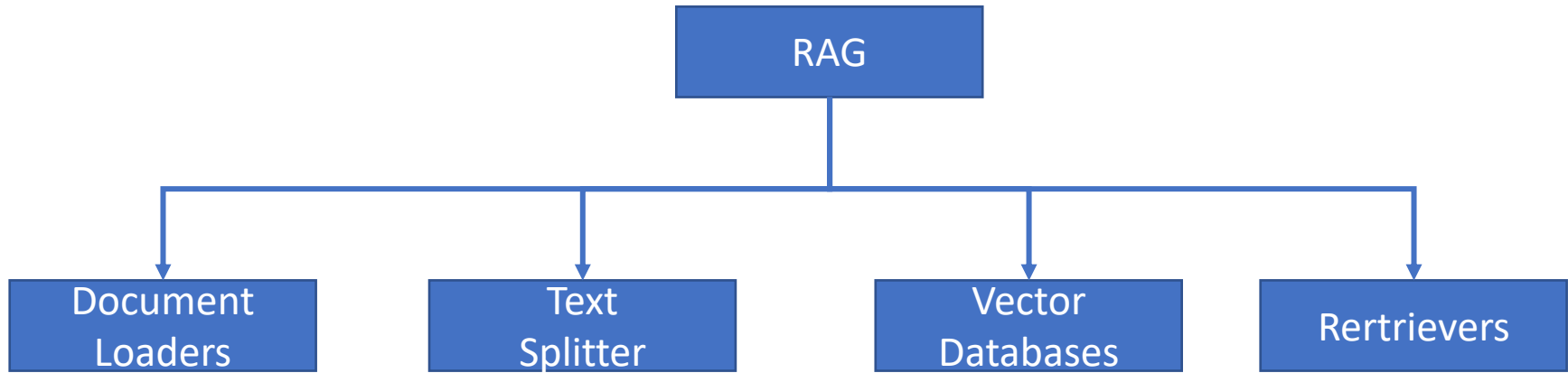
RAG (Retrieval-Augmented Generation)

A technique that combines information retrieval with language generation, where a model retrieves relevant documents from a knowledge base and then uses them as context to generate accurate and grounded responses.

Benefits of using RAG:

- ✓ Use of up-to-date information
- ✓ Better privacy
- ✓ No limit of document size

RAG (Retrieval-Augmented Generation)



Document Loaders in LangChain

Document loaders are components in LangChain used to load data from various sources into a standardized format (usually as Document objects), which can then be used for chunking, embedding, retrieval, and generation.

Document Loader Types

TextLoader	Reads plain text (.txt) files
PyPDFLoader	Loads content from PDF files
WebBaseLoader	Extracts text from web pages (URLs)
CSVLoader	Loads CSV files into Document objects
DirectoryLoader	Loads multiple documents from a directory

TextLoader

- Simple and commonly used document loader
- Reads plain text (.txt) files
- Converts files into LangChain Document objects
- Use Case: Chat logs, scraped text, transcripts, code snippets
- Limitation: Works only with .txt files

PyPDFLoader

- Used to load content from PDF files
- Converts each page into a Document object
- Uses the PyPDF library under the hood
- Limitation: Not great with scanned PDFs or complex layouts
- Best for: Standard text-based PDF documents

PyPDFLoader

Use Cases	Recommended Loader
Simple, clean PDFs	PyPDFLoader
PDFs with tables / columns	PDFPlumberLoader
Scanned / Image PDFs	UnstructuredPDFLoader or AmazonTextractPDFLoader
Need layout and image data	PyMuPDFLoader
Want best structure extraction	UnstructuredPDFLoader

https://docs.langchain.com/oss/python/integrations/document_loaders

DirectoryLoader

- Lets you load multiple documents from a directory (folder)
- Processes all files in a specified folder path
- Automatically converts each file into Document objects
- Use Case: Batch loading documents from a data directory
- Supports: Multiple file types when combined with appropriate loaders

DirectoryLoader

Glob Pattern	What it loads
<code>**/*.txt</code>	All .txt files in all the subfolders
<code>*.pdf</code>	All pdf files in the root directory
<code>data/*.csv</code>	All .csv files in the data/ folder
<code>**/*</code>	All files (any type, all the folders)
<code>**</code>	Recursive search through subfolders

WebBaseLoader

- Loads and extracts text content from web pages (URLs)
- Uses **BeautifulSoup** under the hood to parse HTML
- Extracts visible text from static web content

When to Use:

- Blogs, news articles, or public websites with text-based static content

Limitations:

- Doesn't handle JavaScript-heavy pages well (use **SeleniumURLLoader** for that)
- Loads only static content

CSVLoader

- Used to load CSV files into LangChain Document objects
- Creates one Document object per row, by default
- Preserves column structure and data integrity
- Use Case: Loading tabular data for processing
- Enables: Text-based analysis of structured data

Overview

load()

- Eager Loading
- Loads everything at once
- Immediate execution

lazy_load()

- Lazy Loading
- Loads on demand
- Generator-based

Scope → Generator of load

Choose based on your data size, memory constraints, and processing needs

load() - Eager Loading

Loads everything at once

All documents are loaded into memory immediately when load() is called.

Returns a List

Returns a complete list of Document objects, not a generator.

Immediate Memory Usage

All documents occupy memory from the start - immediate memory footprint.

Access Pattern

Access all documents immediately after calling load().

lazy_load() - Lazy Loading

Loads on demand

Documents are fetched only when needed, one at a time.

Returns a Generator

Returns a generator object that yields Document objects sequentially.

Delayed Memory Usage

Documents are loaded into memory only when accessed - progressive loading.

Access Pattern

Fetch documents as needed using iteration or async patterns.

Eager Loading - Best Use Cases

- ✓ Small number of documents (< 100 documents)
- ✓ You want everything loaded upfront for immediate access
- ✓ All documents fit comfortably in available memory
- ✓ Sequential processing where you need random access to any document
- ✓ Building a complete search index over all documents
- ✓ Training/fine-tuning with fixed dataset

Lazy Loading - Best Use Cases

- ✓ Large number of documents (> 1000 documents)
- ✓ Large document files that consume significant memory
- ✓ Streaming processing (chunking, embedding) without high memory usage
- ✓ Processing documents one at a time sequentially
- ✓ Building pipelines that process infinite or very large datasets
- ✓ Real-time systems where you load data as it arrives

Load vs Lazy Load Comparison

Aspect	Eager Load	Lazy Load
Method	load()	lazy_load()
Return Type	List of Documents	Generator of Documents
Memory Usage	All at once	On demand
Speed	Slower start	Fast start
Best For	Small datasets	Large datasets
Access	Random access	Sequential access

Code Examples

Eager Loading

```
# Load all documents at once
loader = TextLoader("docs/")

# Returns a list
documents = loader.load()

# All docs in memory
for doc in documents:
    print(doc.content)
```

Lazy Loading

```
# Get a generator
loader = TextLoader("docs/")

# Returns a generator
docs_gen = loader.lazy_load()

# Load one at a time
for doc in docs_gen:
    print(doc.content)
```

Memory Usage Profile

load() - Eager Loading



Memory Usage (Immediate)

Time →

High immediate memory demand. All data loaded upfront.

lazy_load() - Lazy Loading



Time →

Incremental memory usage. Data loaded progressively as needed.

How many documents do you have?

< 100 docs

Use `load()`
Eager Loading

> 100 docs

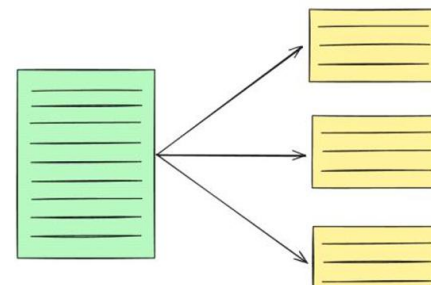
Use `lazy_load()`
Lazy Loading

- Available memory on your system
- How quickly you need to start processing
- Whether you need random access to documents
- Real-time vs batch processing requirements

Text Splitting

The process of breaking large chunks of text (articles, PDFs, HTML pages, books) into smaller, manageable pieces (chunks) that an LLM can handle effectively.

- Overcoming model limitations: Many models have maximum input size constraints
- Downstream tasks: Text splitting improves nearly every LLM powered task
- Optimizing resources: Smaller chunks are more memory-efficient and enable parallelization



Why Text Splitting Matters

1. Overcoming Model Limitations

Many embedding models and language models have maximum input size constraints. Splitting allows us to process documents that would otherwise exceed these limits.

2. Downstream Tasks

Text Splitting improves nearly every LLM powered task

Task	Why Splitting Helps
Embedding	Short chunks yield more accurate vectors
Semantic Search	Search results point to focused info, not noise
Summarization	Prevents hallucination and topic drift

3. Optimizing Computational Resources

Working with smaller chunks of text can be more memory-efficient and allow for better parallelization of processing tasks.

Handles Constraints

Improves Accuracy

Efficient Processing

Why Text Splitting Matters

Key Takeaways:

-  Overcome model input size constraints by splitting large documents into manageable chunks
-  Improve embedding accuracy and semantic search quality with focused, shorter text segments
-  Reduce memory footprint and enable parallel processing for better resource utilization
-  Prevent LLM hallucinations and topic drift in downstream tasks like summarization

Text Splitting Methods

1. Length-Based

Split text into fixed-size chunks

2. Text-Structured

Split based on text delimiters and structure

3. Document-Structured

Split based on document format and sections

4. Semantic Meaning

Split based on semantic similarity and meaning

1. Length-Based Text Splitting

- Divides text into fixed-size chunks
- Simple and straightforward approach
- Useful for uniform chunk sizes
- Example: Split 1000-character document into 500-char chunks
- Trade-off: May split meaningful content across chunks

2. Text-Structured Based Splitting

- Splits based on text delimiters (spaces, newlines, punctuation)
- Preserves logical text boundaries
- Works well with structured text data
- Example: Split on paragraph breaks or sentence boundaries
- Maintains better readability and context

3. Document-Structured Based Splitting

- Splits based on document format and structure
- Respects document hierarchy and sections
- Different logic for different document types (Markdown, HTML, JSON)
- Preserves heading structures and formatting
- Best for: Complex structured documents

4. Semantic Meaning Based Splitting

- Splits text based on semantic similarity and topic coherence
- Uses embeddings to understand meaning
- Keeps semantically related content together
- Most intelligent but computationally expensive
- Best for: Preserving context and meaning across chunks

Key Takeaways

- Document Loaders convert various data sources into standardized LangChain Document objects
- Choose the right loader based on your data source (text, PDF, web, CSV, directory)
- Text Splitting is essential for processing large documents with LLMs
- Select splitting method based on your data structure and semantic requirements
- Together, loaders and splitters form the foundation of LangChain data pipelines